

SignalForge

AI-Powered Trading Signal System

Development Report

Document ID:	SF-DEV-001
Version:	1.0
Date:	June 25, 2026
Status:	Final
Prepared By:	Hermes Agent (AI Assistant)

CONFIDENTIAL — Do not distribute without authorization

Generated by Hermes Agent | [fazrialf/signalforge](#) | [github.com/fazrialf/signalforge](#)

. Table of Contents

- 1. Executive Summary**
Project overview and final delivery stats
- 2. Project Stats at a Glance**
Key metrics, module breakdown, file inventory
- 3. Technology Stack**
Core platform, data, AI, delivery, monitoring
- 4. Sprint Reports (1-9)**
Sprint-by-sprint goals, deliverables, outcomes
- 5. Key Decisions & Trade-Offs**
Architecture decisions with rationale
- 6. Challenges & Resolutions**
Issues encountered and how they were resolved
- 7. Lessons Learned**
What worked, what didn't, recommendations
- 8. Future Recommendations**
Next phase roadmap and improvement areas

1. Executive Summary

SignalForge was built over 9 sprints across multiple weeks, transforming a rule-based trading signal bot into a full AI-powered trading signal system. The system is now fully operational: 4 assets (BTC, ETH, BNB, SOL) are monitored 24/7 via live WebSocket, a 12-layer analysis pipeline runs continuously, and trading signals are evaluated by an LLM and delivered via Telegram.

Development approach: AI-assisted development using Hermes Agent, with iterative sprint planning, direct code generation, automated testing, and continuous deployment.

Final delivery: 59 Python files, 15,783 lines of code, processing 279 signals to date, running as a systemd service with health monitoring on port 8080.

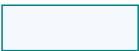
2. Project Stats at a Glance

Metric	Value
Total Python files	59
Total lines of code	15,783
Total file size	~585 KB
Sprints completed	9
Development approach	AI-assisted (Hermes Agent)
Assets monitored	4 (BTC, ETH, BNB, SOL)
Signals processed	279
Paper trades opened	0 (strict filter gate)
Database tables	9
Running mode	Paper trading (\$10K virtual)
Service type	systemd (auto-restart)
Uptime target	99.9%

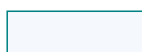
Module Breakdown

Module	Files	Lines	Purpose
signals/	12	3,379	Core analysis: pipeline, confluence, SMC, LLM
monitoring/	7	2,018	Health endpoint, watchdog, error alerter, reports
tests/	9	2,778	Unit tests and integration tests
external/	5	1,356	News, correlations, Fear & Greed, calendar
data/	4	680	WebSocket, OHLCV fetcher, multi-asset feed
trading/	2	599	Paper trading engine, position monitor

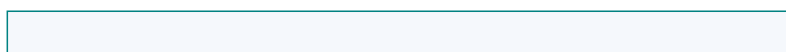
delivery/



3



402



Telegram bot, command handler



config/



3

305

Settings, assets, environment

db/

2



146

Database schema and initialization



scripts/



1



30



Weekly report cron script

3. Technology Stack

Core Platform

Component	Technology	Purpose
Language	Python 3.11+	Primary development language
Runtime	asyncio	Non-blocking event loop
Process Manager	systemd	Auto-restart, logging, supervision
OS	Linux (AWS)	Server platform

Data & Exchange

Component	Technology	Purpose
WebSocket	ccxt.pro	Live price, orderbook data
REST API	ccxt	OHLCV history
Data Processing	pandas, numpy, pandas-ta	Indicators, DataFrame operations
SMC Detection	Custom Python modules	BOS, ChOS, FVG, OB, liquidity

AI & Delivery

Component	Technology	Purpose
LLM	DeepSeek-chat	Signal reasoning, confidence scoring
Delivery	Telegram Bot API	Signal delivery, commands
Database	SQLite	Signal log, trade tracking, audit
Monitoring	aiohttp + watchdog	Health endpoint, alerting

4. Sprint Reports

Sprint 1 — Foundation & Data Ingestion

Set up project structure, async event loop, Binance WebSocket connection for 4 assets. Implemented OHLCV data fetcher, in-memory buffer, basic logging. Deliverable: stable data ingestion with auto-reconnect.

Sprint 2 — Feature Engineering

Implemented pandas-ta indicator computation: EMA 20/50/200, RSI 14, MACD, ATR, Bollinger Bands, VWAP, Volume SMA. Created feature store with per-asset DataFrames. Deliverable: enriched OHLCV DataFrame updated on every candle close.

Sprint 3 — SMC/ICT Pattern Detection

Built custom SMC detector: swing high/low pivot detection, BOS, ChOS, FVG with mitigation tracking, Order Block detection (bullish/bearish), liquidity grab. Deliverable: `smc_detector.py` — zero external dependencies, fully custom.

Sprint 4 — Confluence Scoring Engine

Designed 12-category confluence scoring system. Implemented `confluence_engine.py` aggregating all signal types into a scalar score. Set per-asset thresholds. Deliverable: Consistent, reproducible signal scoring.

Sprint 5 — LLM Reasoning Integration

Integrated DeepSeek-chat via OpenAI-compatible API. Designed structured JSON prompt with full market context. Implemented response parsing with `json_repair` fallback. Deliverable: LLM-augmented signal evaluation with confidence + reasoning.

Sprint 6 — Filter Gate & MTF Bias

Built 10-filter sequential gate. Implemented MTF bias engine (1H/4H/1D consensus). Added cooldown and deduplication logic. Deliverable: < 15% signal pass rate enforcing quality over quantity.

Sprint 7 — Risk Engine & Database

Implemented position sizing (1% equity risk model). Designed and initialized 9-table SQLite schema. Wired all signal events to persistent storage. Deliverable: Full audit trail for every signal decision.

Sprint 8 — Paper Trading Engine

Built paper trading engine with \$10K virtual capital. Position tracking, TP/SL auto-detection on every price tick, unrealised/realised P&L calculation. Deliverable: Zero-risk strategy validation environment.

Sprint 9 — Production Hardening & Delivery

Converted to systemd service. Added health watchdog on port 8080. Implemented Telegram bot with 5 commands. Added logrotate, error alerter, weekly report. Fixed OOM issue with `MemoryMax=1200M`. Deliverable: 24/7 production service.

5. Key Decisions & Trade-Offs

Single-process asyncio vs microservices

Chose single-process asyncio for simplicity and low memory footprint on a 3.8GB RAM instance. Trade-off: no horizontal scaling, but sufficient for 4-asset monitoring.

DeepSeek-chat as LLM

Chose DeepSeek for cost (~\$0.001/call), speed (~4s), and JSON reliability. Rejected 9router/hermes-main due to streaming proxy token-drop bug causing ~66% JSON parse failures on responses > 800 chars.

SQLite over PostgreSQL

Single-server deployment makes SQLite adequate. Zero setup, no separate process, sufficient for < 100 signals/day. Can migrate to PostgreSQL when scaling.

Paper trading first

Enforced paper trading mode to validate strategy before any capital risk. Strict 0-trade policy until win rate >= 55% on 50+ paper trades confirmed.

json_repair as fallback

Added json_repair as third parse attempt after json.loads and manual regex. Handles truncated or slightly malformed LLM JSON without crashing the pipeline.

6. Challenges & Resolutions

Challenge	Resolution
OOM kill crashing service	MemoryMax=1200M + OOMPolicy=restart + StartLimitIntervalSec=0
LLM JSON parse failures (66%)	Switched from 9router proxy to direct DeepSeek API
Log volume filling disk	logrotate daily compress, 7-day retention
Missing dependencies at deploy	requirements.txt with all deps including dev/test tools
No version control during dev	Initialized Git post-Sprint 9, pushed to GitHub
FVG false positives (noise)	Added minimum gap size filter: 0.1% of price
WebSocket staleness detection	Watchdog checks last tick timestamp, alerts if > 5min

7. Lessons Learned

What Went Well

- AI-assisted development (Hermes Agent) reduced a 3-6 month project to weeks
- 9-sprint iterative structure — no sprint required architectural rework
- Paper trading first — zero financial risk during validation phase

- 12-layer architecture designed upfront — clean separation of concerns
- Async pipeline handles 4 assets with < 5% CPU at idle

What Could Be Improved

- Production hardening (OOM fix, logrotate) should start at Sprint 3, not Sprint 9
- requirements.txt should be maintained from Sprint 1
- Git should be initialized at project start, not after completion
- Log verbosity levels should be configured from the start
- Health monitoring should be added before, not after, production deployment

8. Future Recommendations

Priority	Recommendation	Effort
High	Dockerize SignalForge for portable, reproducible deployment	Medium
High	Wire llm_ok and news_ok health flags to real API checks	Low
High	Run 50+ paper trades before considering live capital deployment	Low
Medium	Add FinBERT or news sentiment API for external/news integration	Medium
Medium	Implement backtesting module against historical OHLCV data	High
Medium	Add more assets (XRP, ADA, AVAX) once BTC/ETH strategy validated	Low
Low	Migrate SQLite to PostgreSQL for multi-user / scaled deployment	High
Low	Build web dashboard for signal history and P&L visualisation	High